Mini Project 4: Rebuild Your RAG with LangChain

Justin Garter
IT7075C Applied AI for Cybersecurity, Dr. Zag ElSayed
University of Cincinnati, November 2026

Video: https://youtu.be/Kg6s0j6Pn4k
Project folder: https://github.com/justin-garter/IT7075-coursework/tree/main/module05/langchain_project

Summary

I rebuilt my DFARS, CMMC and ITAR compliance bot from the RAG module with LangChain, on the same five documents and the same eight questions. At the RAG module's settings both versions score the same: 6 of 6 answers found, 1 of 2 refusals held, with Q8 missed. LangChain cut the pipeline to about ten lines, but it chose chunk boundaries and scores I had to dig out to see, and on Q8 it retrieved a chunk that led the model to a confident wrong answer.

Concept check

What problem does LangChain solve? Calling a model API directly means writing the glue yourself: formatting the prompt, sending it, parsing the reply and wiring in retrieval. LangChain gives every one of those steps the same interface, so I can swap the model, vector store or prompt without rewriting the code around it. In this project I switched from the offline stand-in to mistral:7b by changing one object.

What is a prompt template? It's a prompt written once with named placeholders, like {context} and {question}, that the chain fills in on every call. Placeholders keep the instructions fixed while the data changes, so the same tested wording runs for every question and nothing gets pasted together by hand.

What does a prompt, model and parser chain do? The prompt template takes a dictionary of inputs and turns it into chat messages. The model takes those messages and returns an AI message. The output parser pulls the plain text out of that message, so the next step gets a string instead of an object.

What is a retriever? A retriever wraps a vector store behind one call, invoke(question), that returns the most relevant Documents. In a chain it sits on the context branch, retriever | format_docs, so the question goes in and a block of context text comes out, ready for the {context} placeholder.

Branch or parallel? Use a branch when the input decides which single path to take, and parallel when several independent analyses should all run on the same input. A branch fits alert triage: a phishing report goes to a phishing-analysis prompt and a malware alert goes to a malware prompt. Parallel fits indicator enrichment: one IP address goes to a reputation lookup, a WHOIS summary and a log search at the same time, and the results are merged into one report.

1. What I built

A LangChain version of the compliance question-and-answer bot from the RAG module. TextLoader loads each of the five eCFR excerpts, CharacterTextSplitter splits them, Chroma.from_documents embeds them locally with all-MiniLM-L6-v2, a similarity_score_threshold retriever keeps the relevant chunks, and an LCEL chain pipes the context and question through a ChatPromptTemplate into mistral:7b on Ollama at 127.0.0.1:11434. The documents and questions are unchanged: all six my_kb files and goldset.json have the same MD5 hashes as the RAG module copies.

2. What LangChain replaced

Stage	LangChain	RAG module code it made unnecessary
Load and split	TextLoader, CharacterTextSplitter.split_documents()	split_into_chunks() and the file loop in load_and_chunk()
Store	Chroma.from_documents()	embed(), build_collection() and collection.add() with ids, embeddings and metadatas
Retrieve	as_retriever(search_type="similarity_score_threshold")	retrieve(), including the max_distance filter
Prompt	ChatPromptTemplate with {context} and {question}	MY_SYSTEM.format(context=...) and the string assembly in build_context()
Chain	{context, question} | prompt | model | parser	my_ask(), which called retrieve, build_context and generate_local in order

3. Where the framework helped and where it hid something

It helped with plumbing. Source metadata travels with every chunk automatically, the store is one line, and swapping the model is one object. The chain reads as a declaration of the data flow instead of a function I have to trace.

It hid how chunks were cut. CharacterTextSplitter never cuts inside a line, so chunk_size is a target, not a limit. My DFARS cloud-provider paragraph became a 761-character chunk at size 600, and at size 200, 23 of 42 chunks were over the limit. The only sign was a log warning, so I added max_chunk and n_oversized columns to my results to make it visible. Worse, the 100-character overlap did nothing: overlap only carries over whole lines that fit in 100 characters, my regulation paragraphs are longer than that, and none of the 17 pairs of neighboring chunks share any text.

It hid how scores are computed. With the default settings, Chroma measures L2 distance and LangChain converts it to a relevance score with its own formula. That's why "the capital of France" scored negative in Part 7. I set the store to cosine space and passed relevance_score_fn = 1 - distance, so that a score threshold of 0.25 means exactly my old max_distance of 0.75. The retriever also returns Documents with no scores attached, so I had to call similarity_search_with_relevance_scores separately just to print them.

It failed silently once. In one run, the k and threshold sweeps reused a cached store and scored 0 of 6 at every setting, even with the threshold at 0.00, with no error. I couldn't reproduce or explain it, so the final notebook builds a fresh store for every configuration and checks that it holds every chunk before scoring.

All of this matters whenever the number has to be defended. If I'm telling an auditor which regulation text the bot relied on, or tuning a threshold, I need to know exactly how the text was cut and how the score was calculated.

4. The numbers

Each row compares the RAG module (MP3) with LangChain (MP4) at the same setting. A distance and a relevance score run in opposite directions: MP3 kept chunks with a distance at or below the threshold, and MP4 keeps chunks with a score at or above it. Because I used cosine space with score = 1 - distance, a distance of 0.75 and a score of 0.25 are the same cut-off. The like-for-like row is chunk 600, overlap 100, k = 4, distance 0.75 / score 0.25.

Setting	Value (MP3 distance / MP4 score)	MP3 chunks	MP3 found, refused	MP3 missed	MP4 chunks	MP4 found, refused	MP4 missed
chunk size	200	100	5/6, 1/2	Q3 Q8	42	5/6, 1/2	Q4 Q8
chunk size	300	52	6/6, 1/2	Q8	36	6/6, 1/2	Q8
chunk size	400	35	6/6, 1/2	Q8	30	6/6, 1/2	Q8
chunk size	600	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
chunk size	800	17	5/6, 1/2	Q4 Q8	17	6/6, 1/2	Q8
chunk size	1000	13	6/6, 1/2	Q8	14	6/6, 1/2	Q8
chunk size	1500	10	6/6, 1/2	Q8	10	6/6, 1/2	Q8
k	1	22	5/6, 1/2	Q5 Q8	22	5/6, 1/2	Q6 Q8
k	2	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
k	3	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
k	4	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
k	6	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
k	8	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.4 / 0.6	22	2/6, 1/2	Q2 Q4 Q5 Q6 Q8	22	4/6, 1/2	Q5 Q6 Q8
threshold	0.5 / 0.5	22	5/6, 1/2	Q5 Q8	22	5/6, 1/2	Q6 Q8
threshold	0.6 / 0.4	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.65 / 0.35	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.7 / 0.3	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.75 / 0.25	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.8 / 0.2	22	6/6, 1/2	Q8	22	6/6, 1/2	Q8
threshold	0.9 / 0.1	22	6/6, 0/2	Q7 Q8	22	6/6, 0/2	Q7 Q8
threshold	1.0 / 0.0	22	6/6, 0/2	Q7 Q8	22	6/6, 0/2	Q7 Q8

Table 1. RAG module results.csv next to results_langchain.csv. Found is of 6, refused is of 2.

The like-for-like row is identical: 22 chunks in both, 6/6 found and 1/2 refused, with Q8 missed. The counts match even though the chunks don't. MP3 cut every 600 characters regardless of content, while LangChain cut at line breaks.

Where they differ, the chunk boundaries explain it. Whole-paragraph chunks match questions more tightly. Q2's best match moved from a distance of 0.400 to 0.218, and Q6's from 0.488 to 0.373. That's why LangChain keeps 4 of 6 answers at the strictest threshold, where MP3 kept only 2. The single-setting differences are boundary effects too: at chunk 200, MP3 missed Q3 and LangChain missed Q4; at chunk 800, MP3 missed Q4 and LangChain found everything; at k = 1, MP3 missed Q5 and LangChain missed Q6.

What didn't change: the threshold is still the setting that matters most, Q7 still stops being refused once the score threshold drops to 0.10, and Q8 is never refused at any setting in either version.

Found isn't the same as answered. Q4 counts as found in both versions, but in this version mistral replied [LLM] and said the context had no explicit information, even though the 0.8 rule was in the retrieved text.

5. Which version I would build next time

For prototypes, or for work where I expect to swap models, stores or prompts, I'd use LangChain. The whole chain is about ten lines and changing a component doesn't touch the rest. For compliance work, where I may have to show an auditor exactly which text the bot used and why, I'd build it by hand, or at least set the splitter, the distance metric and the score function explicitly the way I did here. In practice I'd probably start with LangChain and replace any piece I need to defend.

6. One question this version got wrong

Q8 asks how many security requirements are in NIST SP 800-171 Revision 3. My documents only cover Revision 2, so the bot should refuse.

```
Q: How many security requirements are in NIST SP 800-171 Revision 3?
   retrieved  score 0.632  [sprs_assessments.md]
   retrieved  score 0.572  [sprs_assessments.md]
   retrieved  score 0.535  [sprs_assessments.md]
   retrieved  score 0.494  [dfars_7012.md]
   answer: [RAG] ... it is mentioned that a summary level score of 110 is expected
   to be achieved, which implies that there are 110 individual requirements in total.
```

Transcript excerpt from the ANSWER AND MEASURE cell of langchain_notebook.ipynb.

The retriever returned all three chunks of sprs_assessments.md. The top one, at 0.632, holds the file heading, "NIST SP 800-171 DoD Assessments and SPRS," and the Basic and High Assessment definitions, which repeat the publication name and the words "Security Requirements." Another of the three holds paragraphs (E), (F) and (2), including "a score of 110 is expected to be achieved." The model took that maximum score for Revision 2 and stated it as the requirement count for Revision 3.

The retrieval mistake is the same one as in the RAG module: the embedding matches the question's topic, NIST SP 800-171 and security requirements, and the words "Revision 3" barely change the vector. What changed is the consequence. In the RAG module, the fixed 600-character cut split this file into four chunks, and the one holding the 110 sentence ranked below the top four, so the model never saw it and gave no number. Here, line-based splitting cut the same file into only three chunks, so all three made the top four, and the model got a number it could misuse. No threshold fixes this, because the chunks for Q8 score higher than the best match for Q5. Catching it would take an answer-level check, such as requiring the answer to quote text that actually states a requirement count.

Generative AI use

I used Claude (Anthropic) to fill in the five practice TODOs, write the Part 8 code and README, troubleshoot the GPU and vector-store problems, and draft this report. I ran every cell myself, checked the results against the RAG module and recorded the video.